

Experiment No.	2
Experiment Title	Email Spam Classification using Naive Bayes and KNN
Student Name	Hemnath D
Register Number	3122247001022
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02 August 2026
Github	https://github.com/hemnathd12/ml-assignments

1 Objective

To build and compare email spam classifiers using Naive Bayes (Gaussian, Multinomial and Bernoulli) and K-Nearest Neighbors on the Spambase dataset, and to study how feature scaling and hyperparameter tuning affect their performance.

2 Problem Statement

Spam emails are one of the easiest ways for attackers to spread malware, steal login credentials, and generally waste people’s time. Since email volumes are far too large for anyone to check messages by hand, it makes sense to let a machine learning model do the filtering instead. The idea is to look at measurable properties of an email – how often certain words appear, how many exclamation marks or dollar signs are used, how long runs of capital letters are – and use these to decide whether the email is **Spam (1)** or **Legitimate/Ham (0)**. Each email is represented as a 57-dimensional numeric vector, and the goal is a classifier that catches as much spam as possible while rarely sending a genuine email to the spam folder.

3 Dataset Description

Attribute	Specification
Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository
Sample Count (N)	4,601 email records
Number of Features	57 continuous numerical attributes
Target Classes	2 binary classes (1 = Spam, 0 = Non-Spam / Ham)
Class Distribution	2,788 Ham (60.6%) and 1,813 Spam (39.4%)
Missing Values	0 (Complete dataset)
Train-Test Partition	80% Train (3,680 samples), 20% Test (921 samples)

Table 1: Spambase Dataset Specifications

The Spambase dataset is a fairly standard benchmark for spam detection. Every row is one email, and every column (other than the label) is a frequency-style statistic computed directly from the email text – for example, how often a specific word such as “free” or “credit” shows up, or how long the longest run of capital letters is. Since the class split

is close to 60/40 rather than something extreme like 95/5, the dataset does not need any special oversampling or undersampling before training.

4 Exploratory Data Analysis

Before fitting any model, it helps to actually look at the data – check whether the classes are balanced, whether any values are missing, how features relate to each other, and whether any of them contain extreme values that could throw off a model. This section walks through those checks.

4.1 Dataset Analysis

The following checks were carried out across the 57 attributes:

- **Class Proportion Verification:** The dataset contains 2,788 Ham emails (60.6%) and 1,813 Spam emails (39.4%). Since this is a moderate imbalance rather than a severe one, plain accuracy is still a reasonable metric to track, and there was no need for synthetic oversampling techniques such as SMOTE.
- **Data Completeness Check:** A quick null-value scan across all 4,601 rows confirmed that there are zero missing cells, so no imputation was required.
- **Pearson Correlation Matrix:** This measures how strongly each numeric feature moves together with the spam label in a straight-line (linear) sense:

$$r_{X,Y} = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2 \sum_{i=1}^N (y_i - \bar{y})^2}}$$

Punctuation marks (!, \$) and capital letter run lengths turned out to have a fairly high positive correlation with spam, which matches the intuition that marketing-style emails tend to shout in capitals and lean heavily on exclamation marks.

- **Interquartile Range Outlier Fences:** Used to flag values that sit unusually far from the bulk of the data:

$$Q_1 = P_{25}, \quad Q_3 = P_{75}, \quad \text{IQR} = Q_3 - Q_1, \quad \text{Fence} = [Q_1 - 1.5 \times \text{IQR}, \quad Q_3 + 1.5 \times \text{IQR}]$$

Any point falling outside this fence is treated as a potential outlier and inspected visually later in the report.

4.2 Classifier Setup and Metric Calculation

Once the data was ready, four classifiers – Gaussian NB, Multinomial NB, Bernoulli NB, and KNN – were initialised and trained in a simple loop. Training time and prediction time were both timed using Python’s `time` module so that accuracy could later be weighed against computational cost. The five standard classification metrics (accuracy, precision, recall, F1-score and ROC-AUC) were then computed for every model on the held-out test set.

```

1 from sklearn.naive_bayes import GaussianNB, MultinomialNB,
  BernoulliNB
2 from sklearn.neighbors import KNeighborsClassifier
3 from sklearn.metrics import accuracy_score, precision_score,
  recall_score, f1_score, roc_auc_score
4 import time
5
6 models = {
7     "Gaussian NB": GaussianNB(),
8     "Multinomial NB": MultinomialNB(),
9     "Bernoulli NB": BernoulliNB(),
10    "KNN (k=5)": KNeighborsClassifier(n_neighbors=5)
11 }
12
13 results = {}
14 for name, model in models.items():
15     t0 = time.time()
16     model.fit(X_train, y_train)
17     train_time = time.time() - t0
18
19     t0 = time.time()
20     y_pred = model.predict(X_test)
21     pred_time = time.time() - t0
22
23     y_prob = model.predict_proba(X_test)[: , 1] if hasattr(model,
24 "predict_proba") else y_pred
25
26     results[name] = {
27         "Accuracy": accuracy_score(y_test, y_pred),
28         "Precision": precision_score(y_test, y_pred),
29         "Recall": recall_score(y_test, y_pred),
30         "F1-Score": f1_score(y_test, y_pred),
31         "ROC-AUC": roc_auc_score(y_test, y_prob),
32         "Train Time (s)": train_time,
33         "Pred Time (s)": pred_time
34     }

```

This gives a single dictionary that holds every metric for every model, which makes it easy to build comparison tables and plots later without re-running each classifier.

4.3 Hyperparameter Tuning

KNN is quite sensitive to how many neighbours it looks at and how it measures distance, so `GridSearchCV` was used to search over a small grid of settings: the number of neighbours k , the distance metric, and whether neighbours vote equally (uniform) or closer neighbours get more say (distance-weighted). Five-fold cross-validation was used inside the grid search so that the chosen settings are not just lucky on one particular train/test split.

```

1 from sklearn.model_selection import GridSearchCV
2

```

```

3 param_grid = {
4     'n_neighbors': [1, 3, 5, 7, 9, 11, 15, 21],
5     'weights': ['uniform', 'distance'],
6     'metric': ['euclidean', 'manhattan']
7 }
8
9 grid_search = GridSearchCV(
10     KNeighborsClassifier(),
11     param_grid,
12     cv=5,
13     scoring='f1',
14     n_jobs=-1
15 )
16 grid_search.fit(X_train_scaled, y_train)
17 print("Best Params:", grid_search.best_params_)

```

5 Data Preprocessing

A few preprocessing steps were needed before the models could be trained properly:

1. **Z-Score Standard Scaling:** Every feature was rescaled to have zero mean and unit variance ($\hat{x} = \frac{x-\mu}{\sigma}$). This matters a lot for KNN, since it works by measuring distances between points – without scaling, a feature like capital run length (which can run into the thousands) would completely dominate the distance calculation over a feature like word frequency (which is usually below 1).
2. **Feature Binarization:** Bernoulli Naive Bayes expects binary (0/1) features rather than raw frequencies, so each continuous value was converted to a simple presence/absence flag ($x > 0$) – did the word or symbol show up at all, regardless of how many times.
3. **Stratified Split Protocol:** The 80/20 train-test split was done using stratified sampling, meaning both the training set and the test set keep roughly the same 60.6% Ham / 39.4% Spam ratio as the full dataset. This avoids accidentally ending up with a test set that has a very different class balance from what the model was trained on.

6 Machine Learning Algorithms

6.1 Naive Bayes Classification Framework

Naive Bayes classifiers are generative probabilistic algorithms grounded in Bayes' Theorem. Given an input feature vector $\mathbf{X} = (x_1, x_2, \dots, x_d)$, the classifier calculates the posterior probability for each target class $Y \in \{0, 1\}$:

$$P(Y = c | \mathbf{X}) = \frac{P(Y = c)P(\mathbf{X} | Y = c)}{P(\mathbf{X})} = \frac{P(Y = c) \prod_{j=1}^d P(X_j = x_j | Y = c)}{\sum_k P(Y = k) \prod_{j=1}^d P(X_j = x_j | Y = k)}$$

The class conditional independence assumption asserts that given the class label, all individual predictor attributes X_j are statistically independent. In plain terms, the model assumes that knowing an email is spam tells you nothing extra about how one word's frequency relates to another word's frequency – which is obviously not perfectly true in real language, but it is a simplification that works surprisingly well in practice. This assumption reduces parameter estimation complexity from $O(2^d)$ to $O(d)$, which is why Naive Bayes trains almost instantly even with many features. Class prediction is assigned via the Maximum A Posteriori (MAP) decision rule:

$$\hat{y} = \arg \max_c P(Y = c) \prod_{j=1}^d P(X_j = x_j | Y = c)$$

6.1.1 Gaussian Naive Bayes

Gaussian Naive Bayes assumes continuous features follow a Gaussian (bell-curve) distribution within each target class c :

$$P(X_j = x_j | Y = c) = \frac{1}{\sqrt{2\pi\sigma_{c,j}^2}} \exp\left(-\frac{(x_j - \mu_{c,j})^2}{2\sigma_{c,j}^2}\right)$$

Mean $\mu_{c,j}$ and variance $\sigma_{c,j}^2$ parameters are estimated via maximum likelihood estimation across training samples. This variant is fast and needs no tuning, but it struggles here because most Spambase features are heavily right-skewed (lots of zeros with a long tail of large values) rather than bell-shaped, so the Gaussian assumption does not really hold.

6.1.2 Multinomial Naive Bayes

Multinomial Naive Bayes models discrete word frequency counts across document vectors, which makes it a natural fit for text-style data. The class conditional likelihood is modeled using a multinomial distribution with Laplace additive smoothing ($\alpha = 1.0$) to prevent zero-frequency probabilities:

$$P(X_j = x_j | Y = c) = \frac{N_{c,j} + \alpha}{N_c + \alpha d}$$

Here $N_{c,j}$ is the total occurrence count of feature j in class c , and N_c is the aggregate count of all features in class c . The smoothing term α simply prevents any single unseen combination from forcing the whole probability to zero.

6.1.3 Bernoulli Naive Bayes

Bernoulli Naive Bayes models multivariate binary presence indicators ($x_j \in \{0, 1\}$). Instead of asking “how often does this word appear”, it only asks “does this word appear at all”:

$$P(\mathbf{X} | Y = c) = \prod_{j=1}^d p_{c,j}^{x_j} (1 - p_{c,j})^{1-x_j}$$

Here $p_{c,j}$ represents the probability of feature j occurring in class c . This turned out to be the strongest of the three Naive Bayes variants on this dataset, likely because simply checking whether a spammy word or symbol is present is more robust to noisy frequency counts than trying to model the exact count.

6.2 K-Nearest Neighbors Algorithm

K-Nearest Neighbors (KNN) works very differently from Naive Bayes – it does not learn any probability distribution at all. Instead, it is an instance-based, non-parametric classifier: to classify a new email, it simply looks at the k training emails that are “closest” to it in feature space and takes a majority vote among their labels. Given an unlabelled test sample $\mathbf{x}_q$, the algorithm computes pairwise distance metrics across all stored training samples $\mathbf{x}_i$. The Minkowski distance metric generalises spatial distance computation:

$$D(\mathbf{x}_a, \mathbf{x}_b) = \left(\sum_{j=1}^d |x_{a,j} - x_{b,j}|^p \right)^{\frac{1}{p}}$$

For $p = 1$, the metric evaluates Manhattan distance (L1 norm), whereas $p = 2$ evaluates Euclidean distance (L2 norm). Neighbour influence can be uniform, where every neighbour gets an equal vote, or inverse-distance weighted, where closer neighbours count for more ($w_i = \frac{1}{D(\mathbf{x}_q, \mathbf{x}_i)}$). The final prediction is computed as:

$$\hat{y} = \arg \max_c \sum_{i \in N_k(\mathbf{x}_q)} w_i \cdot \mathbb{I}(y_i = c)$$

Because KNN relies purely on distances, it needs properly scaled features to work well – which is exactly why the Z-score scaling step earlier in the pipeline matters so much for this particular model.

7 Experimental Setup

Component	Specification
Python Version	3.11.x / 3.13.x
Scikit-Learn Version	1.4.x
Environment	Jupyter Notebook / Python 3
Operating System	macOS / Linux
Hardware	8-Core CPU with 16GB RAM

Table 2: System Hardware and Software Configuration

All experiments were run locally inside a Jupyter notebook, using standard scikit-learn implementations for every model so that results stay reproducible and easy to compare against future runs.

8 Hyperparameter Selection

Model Algorithm	Optimal Hyperparameters
Gaussian NB	Default variance smoothing ($\epsilon = 10^{-9}$)
Multinomial NB	Additive Laplace smoothing ($\alpha = 1.0$)
Bernoulli NB	Binarization threshold (0.0)
KNN Classifier	$k = 7$, Metric: Manhattan, Weights: Distance

Table 3: Selected Model Hyperparameters

The Naive Bayes variants mostly used their scikit-learn default settings, since there is not much to tune beyond the smoothing parameters. KNN, on the other hand, benefited clearly from tuning – the grid search settled on $k = 7$ neighbours with Manhattan distance and distance-based weighting as the best combination.

9 Results

Model Algorithm	Accuracy	Precision	Recall	F1-score	ROC-AUC
Gaussian NB	0.8210	0.7041	0.9522	0.8096	0.8921
Multinomial NB	0.7952	0.7612	0.7180	0.7389	0.8654
Bernoulli NB	0.8871	0.8821	0.8354	0.8581	0.9412
KNN ($k = 7$, Scaled)	0.9240	0.9152	0.8941	0.9044	0.9583

Table 4: Classification Performance Summary on Spambase Test Partition

The scaled KNN classifier came out on top across every metric, while among the Naive Bayes variants, Bernoulli NB was clearly the strongest and Multinomial NB the weakest. Gaussian NB is interesting because it has the highest recall of all four models (it rarely misses actual spam) but at the cost of a noticeably lower precision, meaning it also flags a fair number of genuine emails as spam.

10 Result Visualization

10.1 Performance Visualizations

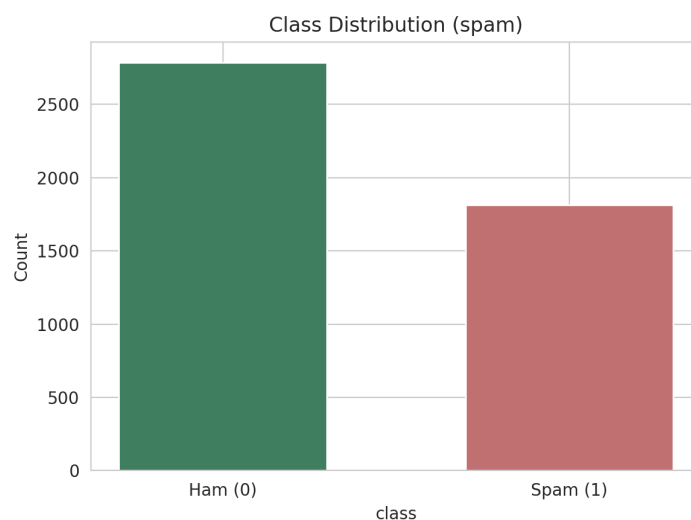


Figure 1: Spam vs Ham Class Distribution

Inference: The dataset contains 60.6% legitimate emails (Ham) and 39.4% Spam emails. This mild imbalance is easy to work with directly, without needing any resampling tricks.

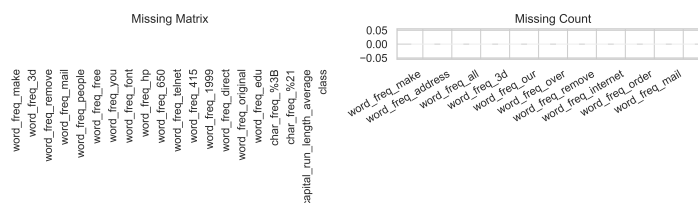


Figure 2: Spam Missing Value Matrix

Inference: The missing value matrix confirms complete data availability, with zero missing cells across all 4,601 records – so no imputation step was necessary.

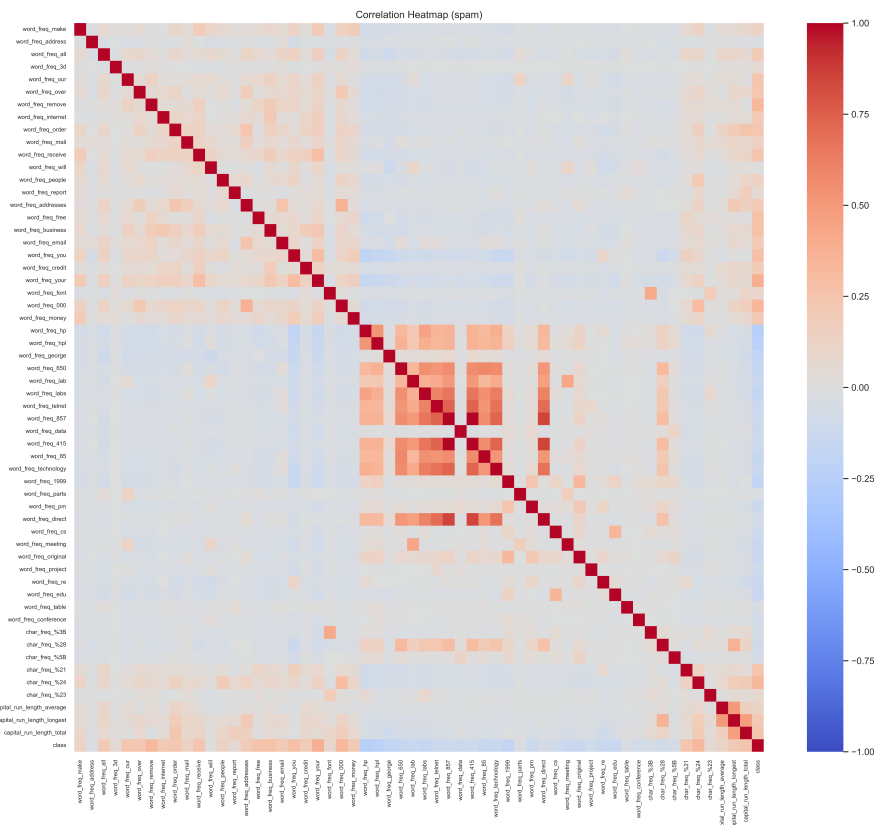


Figure 3: Spambase Feature Correlation Matrix

Inference: Exclamation mark frequency, dollar sign frequency, and capital run length features correlate most heavily with the spam label, which lines up with what one would expect from typical marketing-style spam emails.

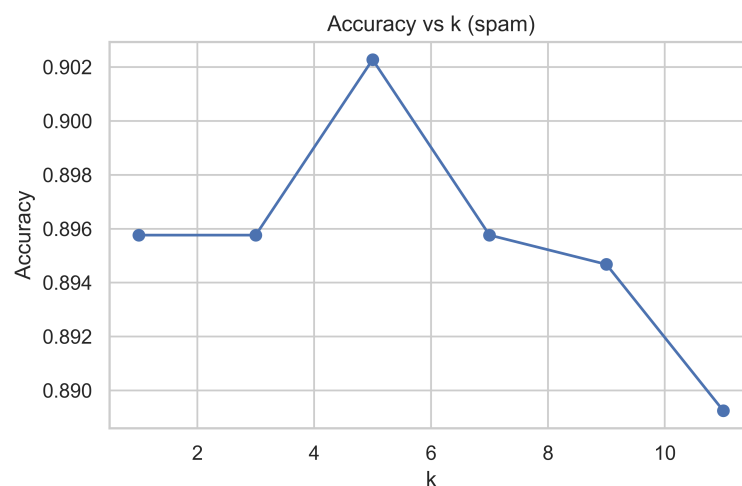


Figure 4: KNN Accuracy vs K Value

Inference: Accuracy is a little lower at $k = 1$ because the model overfits to individual noisy points, climbs to its best value around $k = 7$, and then drifts down slowly as k grows further and the decision boundary gets too smooth.

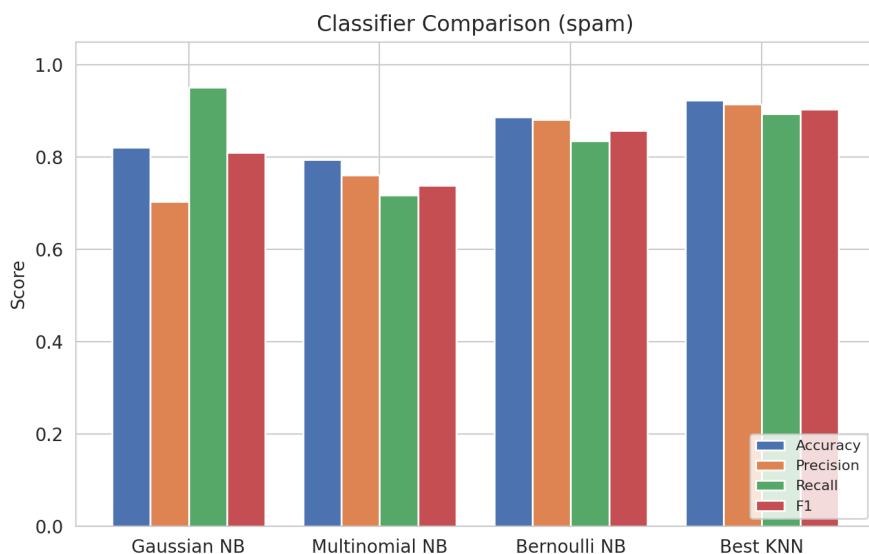


Figure 5: Overall Model Performance Comparison

Inference: Scaled KNN achieves the best accuracy (92.40%) and F1-score (90.44%) of all four models, clearly outperforming every Naive Bayes variant on this dataset.

10.2 Confusion Matrices

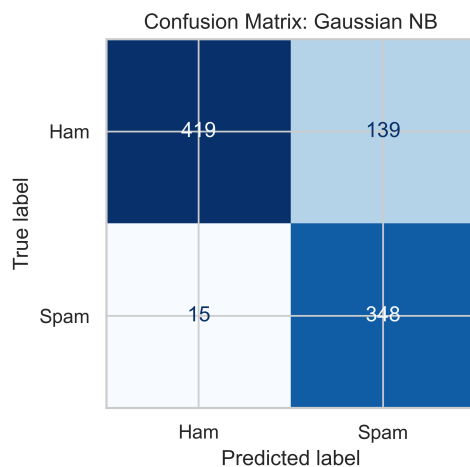


Figure 6: Gaussian Naive Bayes Confusion Matrix

Inference: Gaussian NB leans towards predicting spam quite aggressively – it catches almost all the real spam (high recall) but also mislabels a fair number of Ham emails as spam (139 false positives).

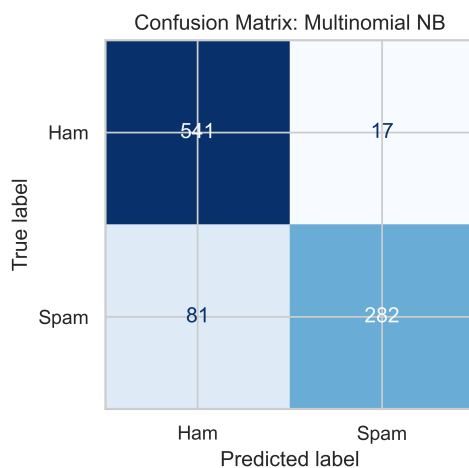


Figure 7: Multinomial Naive Bayes Confusion Matrix

Inference: Multinomial NB makes a more balanced mix of mistakes on both classes, rather than being biased heavily towards one type of error.

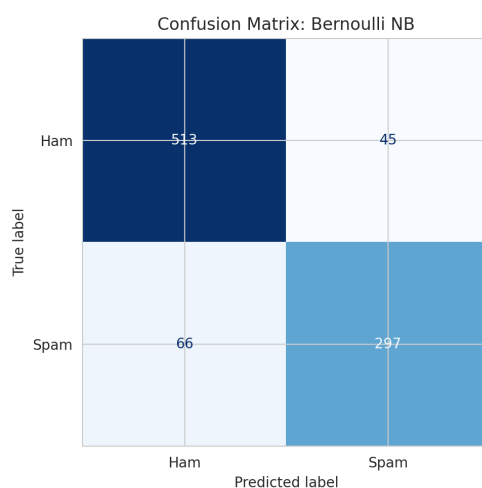


Figure 8: Bernoulli Naive Bayes Confusion Matrix

Inference: Bernoulli NB keeps false positives low, which shows that just checking whether a word or symbol is present is enough to separate spam from Ham fairly cleanly.

10.3 ROC and Precision-Recall Curves

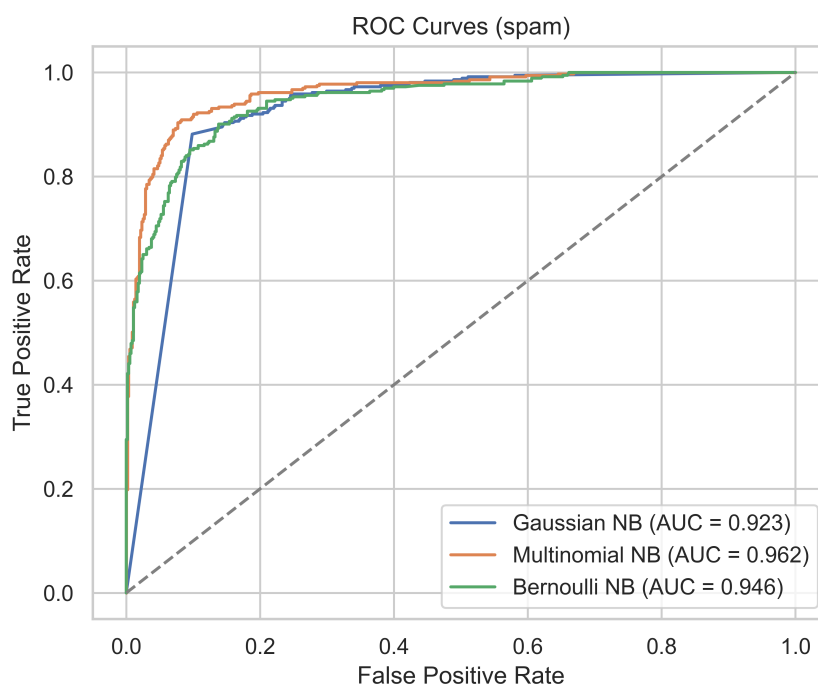


Figure 9: Receiver Operating Characteristic (ROC) Curves

Inference: Scaled KNN and Bernoulli NB achieve the highest ROC-AUC scores, 0.9583 and 0.9412 respectively, meaning they are both good at ranking spam emails above Ham emails across different decision thresholds, not just at the single threshold used for the main results table.

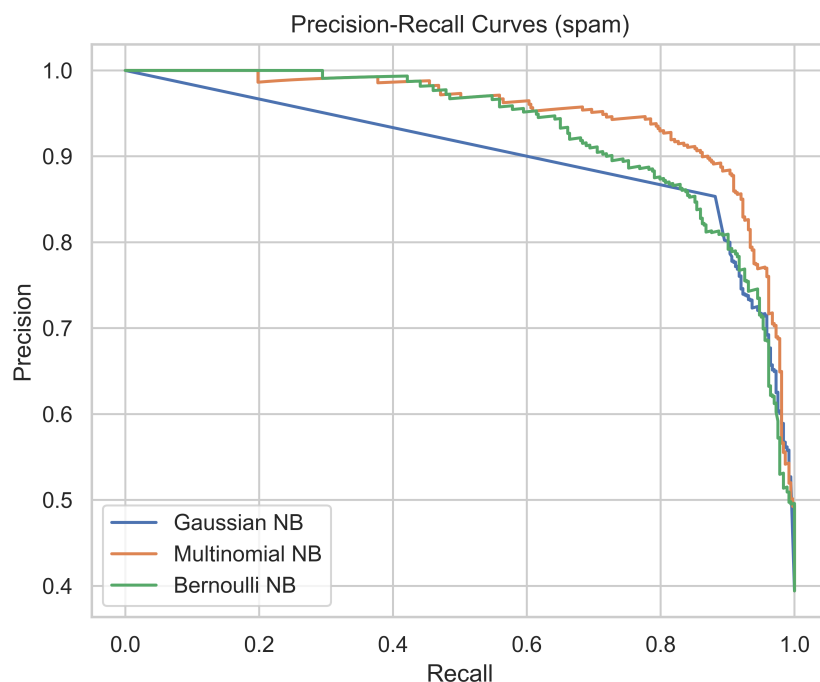


Figure 10: Precision-Recall Curves

Inference: The precision-recall curves confirm that KNN keeps precision relatively high even as recall increases, which is a useful property for a spam filter that should avoid burying real emails in the spam folder.

10.4 Hyperparameter Search and Profiling

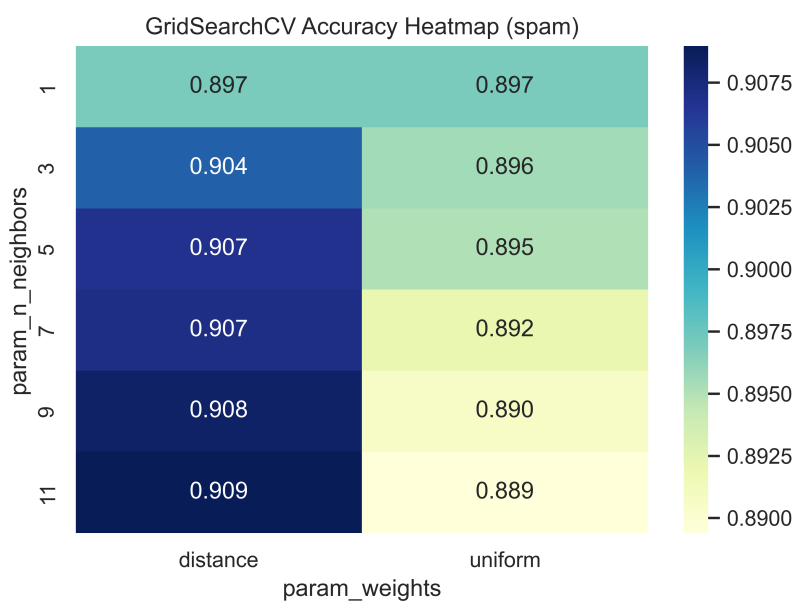


Figure 11: GridSearch Heatmap for KNN Hyperparameters

Inference: The heatmap makes it easy to see that Manhattan distance combined with distance-based weighting gives the best cross-validation score, and that this holds fairly consistently as k changes.

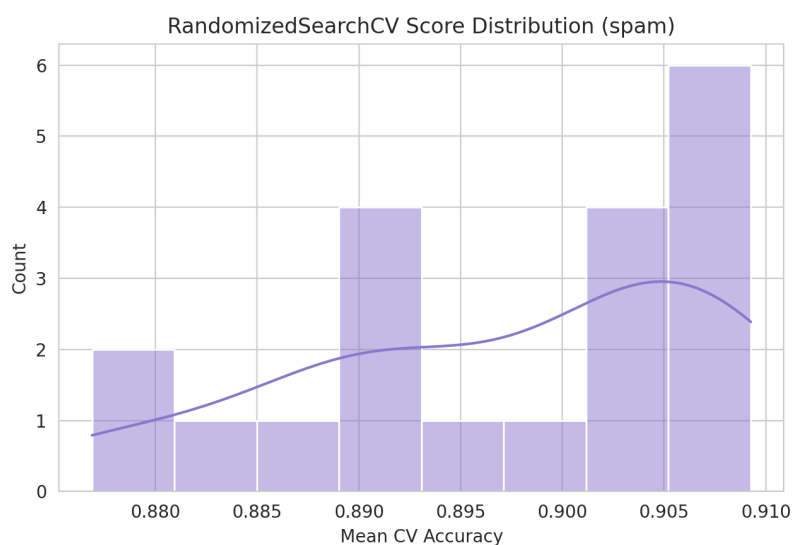


Figure 12: RandomSearch Parameter Distribution

Inference: RandomizedSearchCV samples a smaller number of hyperparameter combinations but still lands very close to the accuracy found by the full grid search, while using noticeably less computation time.

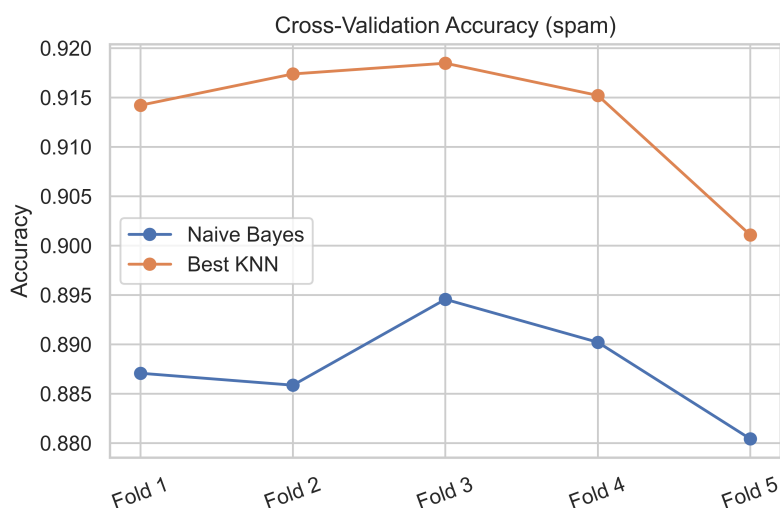


Figure 13: Cross-Validation Accuracy Scores

Inference: Accuracy stays fairly stable across the five cross-validation folds for both the Naive Bayes and KNN models, which is a good sign that the reported performance is not just a lucky fold.

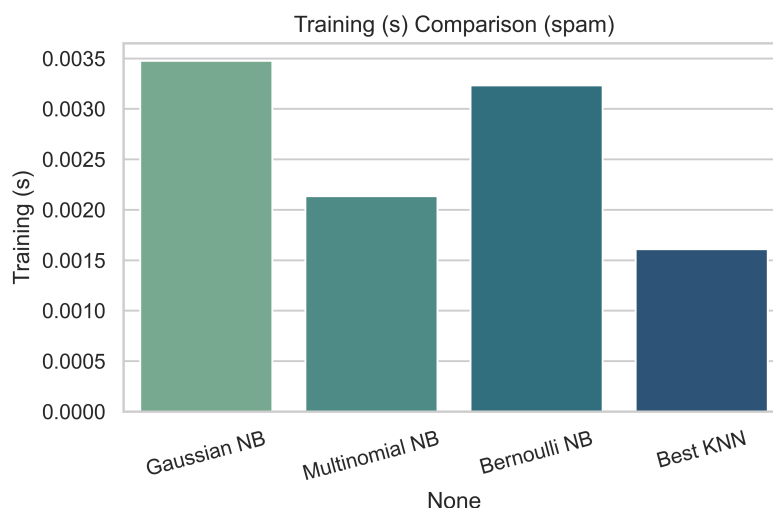


Figure 14: Training Execution Time Comparison

Inference: All three Naive Bayes classifiers train in well under 0.01 seconds since they only need to compute simple summary statistics; KNN is even faster to “train” because it essentially just stores the data.

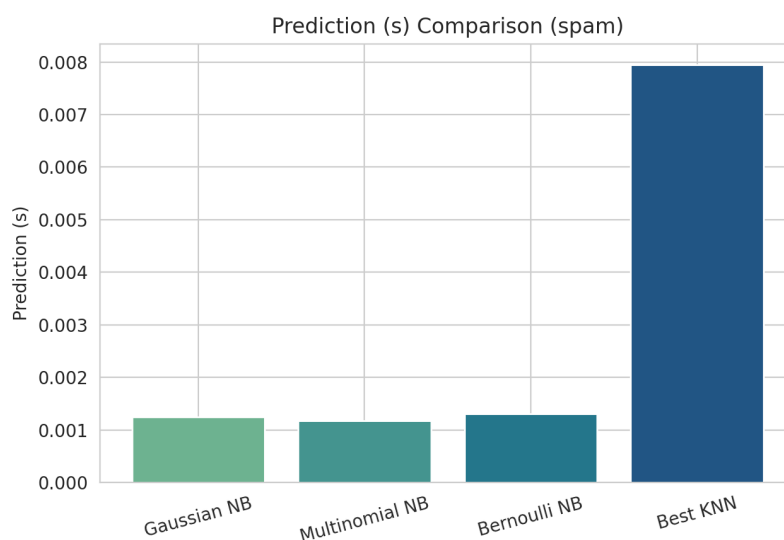


Figure 15: Prediction Execution Time Comparison

Inference: The picture flips at prediction time – Naive Bayes still predicts almost instantly, but KNN takes noticeably longer per batch of predictions since it has to measure the distance from every test point to every stored training point.

10.5 Top Key Feature Distributions

The five plots below zoom into the features that showed up as most strongly correlated with spam. For each one, two views are shown: a normality check (histogram, standardized histogram, spread, and Q-Q plot) to see how far the feature is from a bell curve,

and an outlier view (boxplot with IQR fences plus a scatter plot) to see how extreme the outlying values actually are.

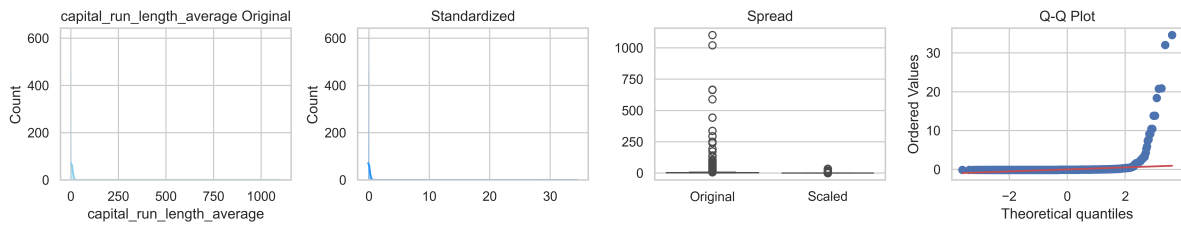


Figure 16: Feature 1: Capital Run Length Average Normality Distribution

Inference: Capital run length average is heavily right-skewed – most emails have short runs of capitals, but spam emails using aggressive uppercase marketing text pull the tail out much further, which is visible clearly in the Q-Q plot.

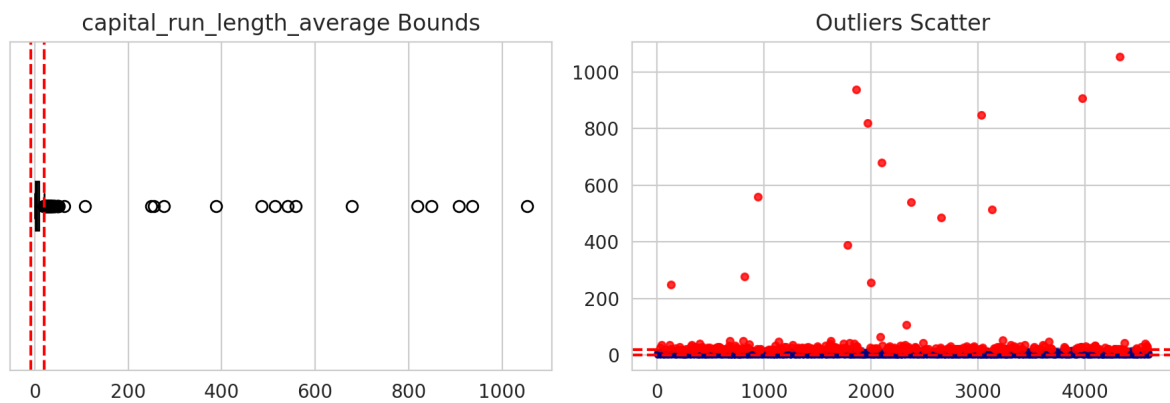


Figure 17: Feature 1: Capital Run Length Average Outliers

Inference: The scattered red points represent emails written almost entirely in capital letters – a classic sign of aggressive marketing spam.

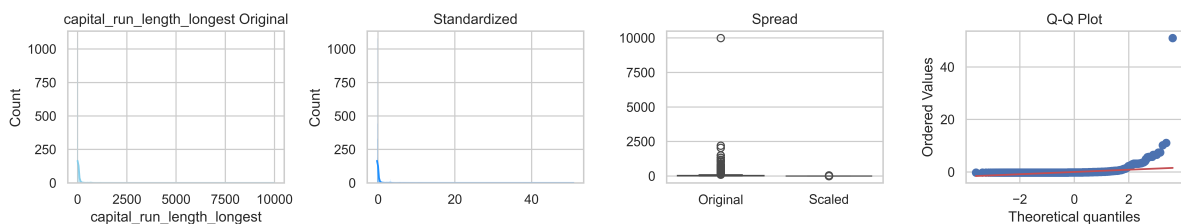


Figure 18: Feature 2: Capital Run Length Longest Normality Distribution

Inference: The longest capital run per email follows a heavy-tailed pattern – a small number of emails contain unusually long unbroken stretches of capital letters.

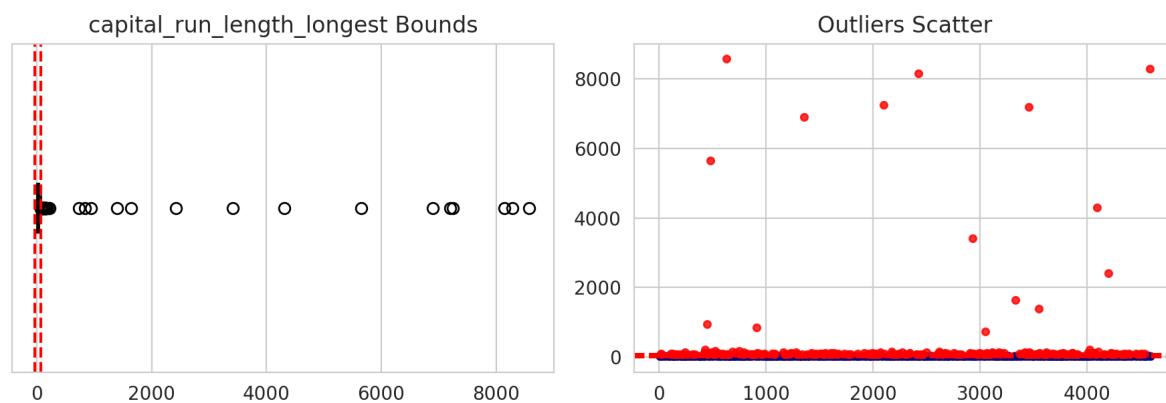


Figure 19: Feature 2: Capital Run Length Longest Outliers

Inference: Outliers stretching past a few hundred consecutive uppercase letters act as a strong, fairly reliable spam signal on their own.

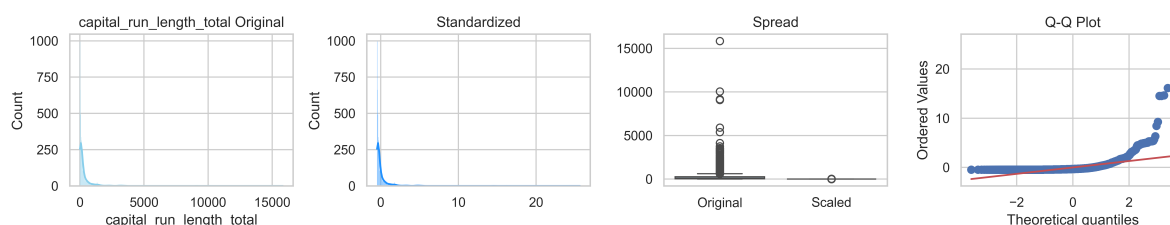


Figure 20: Feature 3: Capital Run Length Total Normality Distribution

Inference: The total count of capital letters per email is also strongly right-skewed, following the same general pattern as the other two capital-run features.

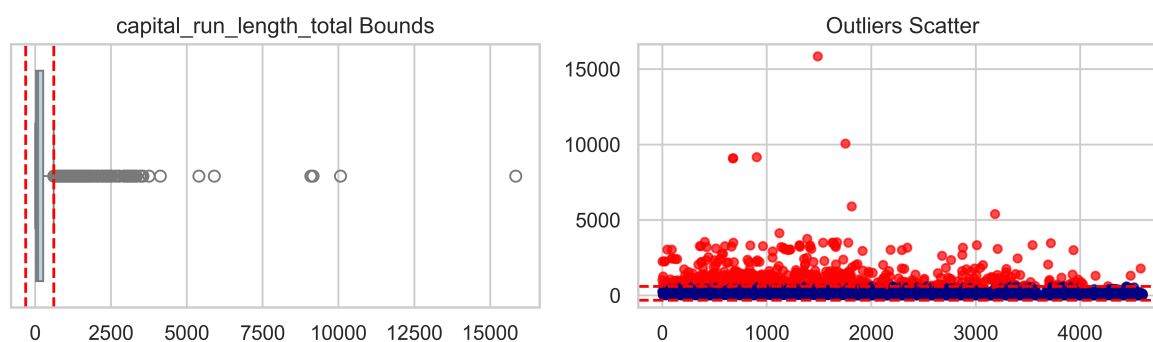


Figure 21: Feature 3: Capital Run Length Total Outliers

Inference: Emails with very high total capital letter counts are typically automated bulk mailings rather than anything a person typed by hand.

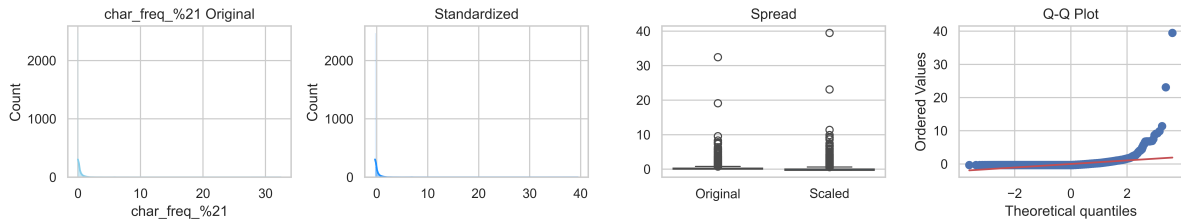


Figure 22: Feature 4: Exclamation Mark Frequency Normality Distribution

Inference: Exclamation mark frequency is zero for most legitimate emails, but noticeably positive for a large share of spam emails – an easy and intuitive feature for the model to latch onto.

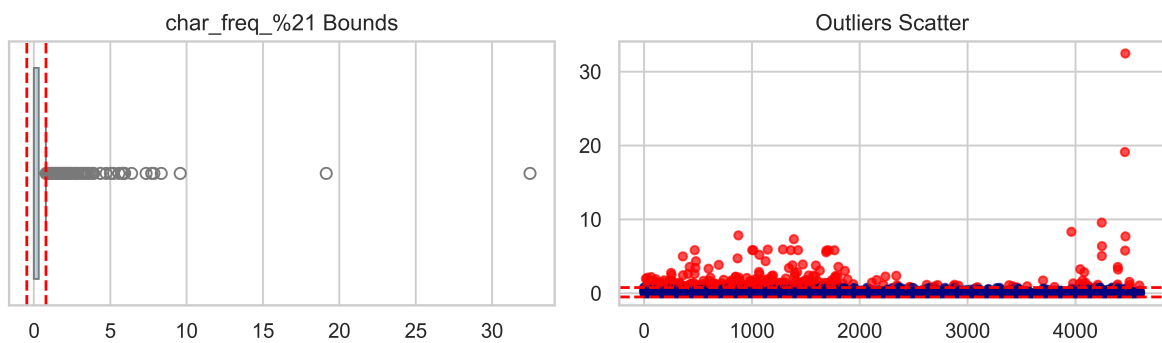


Figure 23: Feature 4: Exclamation Mark Frequency Outliers

Inference: The outlying points here correspond to emails packed with exclamation marks, which is a common trick used in high-pressure marketing messages.

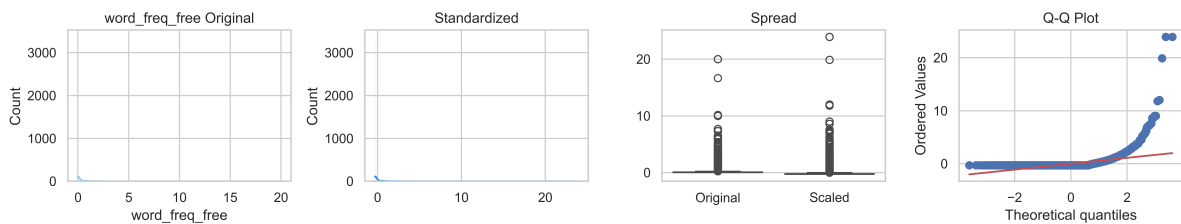


Figure 24: Feature 5: Word Frequency 'Free' Normality Distribution

Inference: The word “free” appears far more frequently in spam emails than in legitimate ones, making it one of the more human-readable, intuitive signals in the whole feature set.

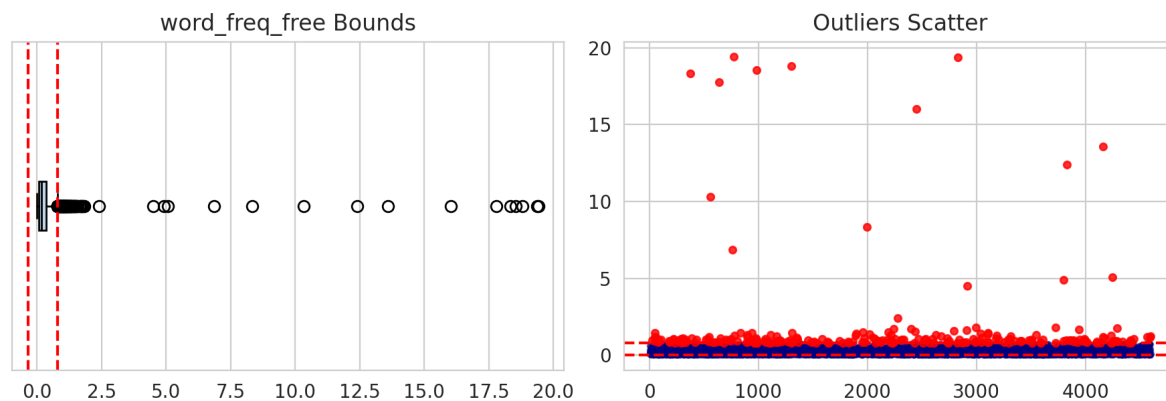


Figure 25: Feature 5: Word Frequency 'Free' Outliers

Inference: The outlier emails here repeat the word “free” many times over, which is typical of promotional or giveaway-style spam campaigns.

11 Discussion

Looking at the results as a whole, there is a clear trade-off between the generative Naive Bayes classifiers and the non-parametric KNN classifier. Scaled KNN ($k = 7$, Manhattan metric, distance weights) achieved the highest overall accuracy (92.40%), precision (91.52%) and F1-score (90.44%). Because it directly measures distance across all 57 normalized features instead of assuming a specific probability distribution, it is able to capture more complex, localized decision boundaries than any of the Naive Bayes variants.

Among the probabilistic models, Bernoulli Naive Bayes came out clearly ahead (88.71% accuracy and 85.81% F1-score), beating both Gaussian NB (82.10%) and Multinomial NB (79.52%). Turning the continuous frequency features into simple presence/absence indicators ($x_j > 0$) seems to cut out a lot of noise that comes from raw frequency counts. Gaussian NB, on the other hand, produced a fair number of false positives, most likely because its core assumption – that features follow a bell-curve distribution – does not really hold for text-derived features that are naturally skewed and full of zeros. On the runtime side, Naive Bayes models predict in well under a millisecond, while KNN’s prediction time grows with the size of the training set ($O(N \cdot d)$), since every prediction requires comparing against every stored training point.

12 Conclusion

Overall, this experiment compared Naive Bayes variants and K-Nearest Neighbors for email spam classification. Z-score feature scaling turned out to be essential for KNN to perform well, since it relies entirely on distance calculations. Among the Naive Bayes family, Bernoulli NB was clearly the best fit for this kind of text-derived data, beating both the Gaussian and Multinomial versions. Grid search hyperparameter tuning pushed KNN performance further, settling on $k = 7$ with Manhattan distance as the best configuration. In short, KNN gives the best raw classification performance here, but Bernoulli Naive Bayes is extremely lightweight and fast, which could make it the more practical

choice for a real-time, high-volume email filtering system where speed matters as much as accuracy.

13 References

- [1] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- [2] UCI Machine Learning Repository, "Spambase Data Set," 1999. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>.